
Teaching the X2 Humanoid to Walk

Training a walking controller in simulation and running it on the real robot

Baaqer Farhat **Mahdi Taheri**

AgiBot X2 MuJoCo mjlab Reinforcement learning

Objective

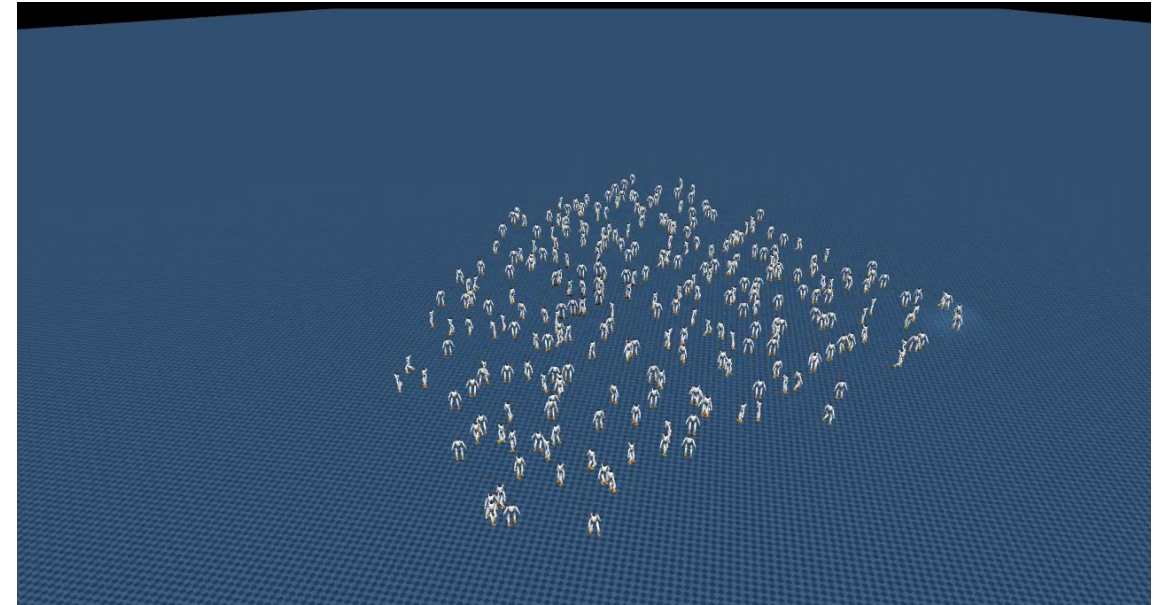
- Learn a velocity conditioned walking controller for the 31 DoF X2 humanoid.
- Track a commanded forward, lateral, and yaw velocity while keeping the base stable.
- Train end to end in simulation with reinforcement learning, then transfer to hardware.
- The gait emerges from the reward objective. There are no reference trajectories or scripted state machines.

The control policy

- A feedforward network mapping proprioceptive state to joint position targets at 50 Hz.
- **Observation:**
 - Base angular velocity and projected gravity.
 - Joint positions and velocities, and the previous action.
 - The velocity command driving the gait.
- **Action:**
 - A position target per joint, tracked by the actuator PD loops.
- Actor and critic share the observation. The critic additionally sees base linear velocity as a privileged, training only signal.

Simulation and training setup

- Training runs in MuJoCo Warp through mjlab, entirely on a single GPU.
- Roughly 4096 environments step in parallel, each an independent robot on its own terrain.
- One shared actor and critic collect experience from all environments at once.
- This yields billions of simulated timesteps within a few hours of wall clock training.



A subset of the parallel environments stepping simultaneously.

Learning formulation

Stochastic policy over joint targets

$$a_t \sim \pi_{\theta}(\cdot | s_t)$$

Actions define PD position setpoints

$$q^{\text{target}} = q^{\text{default}} + \alpha \odot a_t$$

Reward is a weighted sum of shaped terms

$$r_t = \sum_k w_k r_k(s_t, a_t)$$

Velocity tracking via an exponential kernel

$$r_{\text{vel}} = \exp\left(-\frac{\|v^* - v\|^2}{\sigma^2}\right)$$

PPO maximizes the clipped surrogate

$$L(\theta) = \mathbb{E}\left[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t)\right]$$

On policy optimization: rollouts are batched across all environments, advantages estimated with GAE, and the policy updated by PPO with a clipped surrogate and entropy regularization.

Reward design

Reward term	Purpose
Linear and angular velocity tracking	Match the commanded base velocity
Base orientation and height	Penalize torso tilt and unwanted vertical motion
Foot clearance and air time	Enforce distinct swing and stance phases
Action rate and torque	Regularize toward smooth, efficient motion
Foot slip and contact impact	Penalize sliding and hard landings
Termination on fall	End and penalize the episode when the base drops

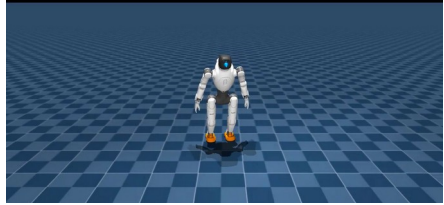
Robustness and transfer

- Velocity curriculum: the command range widens as tracking performance improves.
- Domain randomization: ground friction, added base mass, and actuator gains vary per environment.
- External perturbations: random base pushes force the policy to learn balance recovery.
- Together these reduce the simulation to reality gap and prevent overfitting to one exact setup.

Command conditioned locomotion

- The policy is conditioned on a velocity command: forward, lateral, and yaw rate.
- A single network produces forward, lateral, and turning gaits by varying that command.
- At deployment we stream a fixed command, for example 0.3 m/s forward, to drive walking.

Policy progression over training



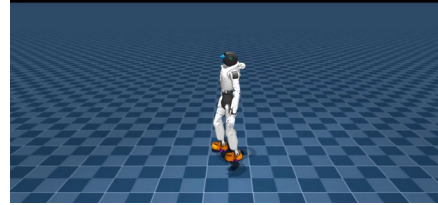
Iteration 0

Untrained, collapses immediately



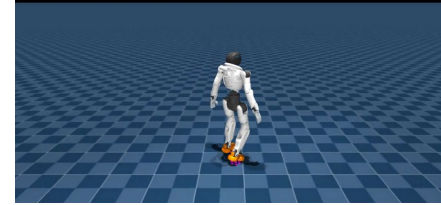
Iteration 100

Unstable, fails to hold balance



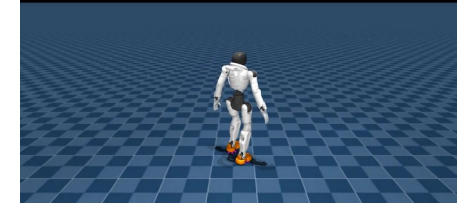
Iteration 3000

Emerging forward stepping



Iteration 6000

Consistent locomotion



Iteration 10000

Stable, converged gait

Deterministic rollouts of saved checkpoints under a fixed forward command.

Deployment on hardware

- The trained actor is exported to a compact model and runs with numpy only on the robot.
- The deployment policy drops base linear velocity from its observation, since it is not measurable on hardware. The critic used it only in training.
- Inference runs at 50 Hz over ROS 2, and the onboard PD loops track the joint targets.
- The velocity command is streamed live, so the same policy walks in any direction on the real robot.

Hardware demo video

insert clip here

Policy running on the physical X2.